BÁO CÁO BÀI TẬP NHÓM

DỰ ĐOÁN GIÁ NHÀ HÀ NỘI BẰNG HỌC MÁY

NHÓM 10
Môn học: Python
Giảng viên: Phạm Danh Tuyên

Thành viên	MSSV
Võ Quang Huy	B24DCKH063
Nguyễn Hữu Nam	B24DCKH092
Lê Đình Phong	B24DCKH098

Hà Nội, tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày hệ thống dự đoán giá chào bán nhà ở Hà Nội từ 657.691 mẫu sau làm sạch và khử trùng. Pipeline chia dữ liệu theo nhóm tin trùng, fit toàn bộ bước tiền xử lý trên train và chọn mô hình bằng GroupKFold 5 fold. Random Forest được chọn với Test R^2 0,711, MAE 5,12 tỷ đồng, Median APE 17,4% và 55,9% dự đoán lệch không quá 20%. Ứng dụng Streamlit cung cấp EDA, dự đoán giá và báo cáo sai số theo phân khúc.

1. Mô tả bài toán, mục tiêu và chức năng chính

1.1 Bài toán

Giá bất động sản phụ thuộc đồng thời vào diện tích, vị trí, loại nhà và nhiều yếu tố khác, nên người mua và người bán thường khó ước lượng một mức giá hợp lý. Đề tài giải bài toán **hồi quy**: dự đoán **giá chào bán** (đơn vị tỷ VNĐ) của một bất động sản tại Hà Nội từ 8 thông tin đầu vào: diện tích, số phòng ngủ, số phòng tắm, số tầng, chiều rộng mặt tiền, loại bất động sản, quận/huyện và phường/xã.

1.2 Mục tiêu

- Xây dựng pipeline huấn luyện **không rò rỉ dữ liệu** và đánh giá trung thực trên tập test độc lập.
- So sánh ba mô hình hồi quy (Ridge, Random Forest, Gradient Boosting) và chọn mô hình tốt nhất bằng cross-validation.
- Báo cáo sai số bằng các chỉ số người dùng dễ hiểu (sai lệch phần trăm trung vị, tỷ lệ dự đoán lệch không quá 20%), không chỉ R^2.
- Cung cấp ứng dụng web để nhập thông tin một căn nhà và nhận giá ước tính.

1.3 Chức năng chính của sản phẩm

Sản phẩm gồm một chương trình huấn luyện (`python src/train_model.py`) và một ứng dụng Streamlit với ba tab:

Tab	Chức năng
Tổng quan dữ liệu	Phân phối giá, giá trung vị theo quận, giá theo loại bất động sản, biểu đồ diện tích so với giá (EDA trên mẫu 50.000 dòng)
Dự đoán giá	Nhập thông tin căn nhà, nhận giá ước tính cùng khoảng tham khảo (giá dự đoán nhân 1 ± MAPE)
Đánh giá mô hình	• Trình bày dữ liệu đã lọc, cách chia tập và chống rò rỉ • Bảng so sánh mô hình • Sai số theo khoảng giá, loại bất động sản, quận/huyện • Mức độ quan trọng của đặc trưng • Giải thích các chỉ số

2. Phân tích và lựa chọn giải pháp

2.1 Dữ liệu

Nguồn dữ liệu: tinixai/vietnam-real-estates trên Hugging Face, gồm 10 file Parquet.

Chương trình lọc các tin của Hà Nội ngay khi đọc và chỉ lấy các cột cần dùng. Các tin được đăng từ 01/06/2025 đến 30/03/2026.

Bước	Số dòng
Tin thô của Hà Nội	1.140.542
Sau khi làm sạch theo quy tắc cố định	875.967 (loại 264.575 dòng)
Sau khi chia nhóm và khử trùng	657.691 (loại 218.276 tin trùng)
Tập train / tập test	526.152 / 131.539

Quy tắc làm sạch được áp dụng độc lập trên từng dòng (không học thống kê từ toàn bộ dữ liệu nên không gây rò rỉ):

- Lọc bỏ dòng thiếu giá, diện tích, loại bất động sản hoặc quận
- Loại bỏ loại "Đất" vì bộ đặc trưng hiện tại chưa phù hợp để định giá đất trống; các biến phòng và tầng không áp dụng cho nhóm này.
- Lọc các lỗi nhập liệu rõ ràng
- Một số quy tắc điển hình và số dòng bị gắn cờ (một dòng có thể vi phạm nhiều quy tắc)

Quy tắc	Số dòng
Giá không dương	23.639
Đơn giá dưới 3 triệu/m²	25.520
Diện tích lớn hơn 2.500 m²	1.385
Mặt tiền ngoài khoảng 1–100 m	348
Mặt tiền lớn hơn diện tích	325
Đơn giá trên 4 tỷ/m²	193

2.2 Các quyết định thiết kế chính

Vấn đề	Giải pháp	Lý do
Giá lệch phải, trải từ vài tỷ đến hàng trăm tỷ	Huấn luyện trên `log1p(giá)`, đổi về tỷ VNĐ bằng `expm1`	Giá thay đổi theo kiểu nhân, thang log ổn định hơn
Một căn đăng nhiều lần	Tạo khóa nhóm từ đặc trưng, vị trí và giá; chia train/test theo nhóm (GroupShuffleSplit), sau đó mới khử trùng từng tập; CV dùng GroupKFold	Tránh để cùng một căn nằm ở cả train lẫn test
Quận, phường có nhiều giá trị	Mã hóa bằng giá trung vị của quận và phường, tính theo kiểu out-of-fold; dự phòng phường, rồi quận, rồi toàn train	Giữ thông tin vị trí mà không để nhãn của chính dòng đó rò vào đặc trưng

Vấn đề	Giải pháp	Lý do
Thiếu số phòng, số tầng, mặt tiền	Điền trung vị của train và thêm cột chỉ báo thiếu	Giữ lại dòng thiếu thông tin thay vì loại bỏ
Chọn mô hình	Chọn theo R^2 trung bình của CV 5-fold trên train; tập test chỉ dùng để báo cáo	Không chọn mô hình dựa trên tập test
Triển khai	Sau khi đánh giá, huấn luyện lại mô hình tốt nhất trên toàn bộ dữ liệu đã khử trùng	Dùng hết dữ liệu cho mô hình đưa vào app

Tập đặc trưng cuối cùng có 45 cột: 5 biến số, 4 cột chỉ báo thiếu, 2 cột mã hóa giá quận/phường, 4 loại bất động sản và 30 quận (one-hot).

2.3 Lựa chọn mô hình

Nhóm so sánh ba mô hình:

- RidgeCV (mô hình tuyến tính có điều chuẩn, dùng làm mốc so sánh),
- **Random Forest** (200 cây, độ sâu tối đa 12)
- **Gradient Boosting** (200 cây, độ sâu 4, learning rate 0,1)

Hai mô hình cây học được quan hệ phi tuyến và tương tác giữa vị trí, loại nhà và diện tích mà mô hình tuyến tính khó biểu diễn.

Bài học từ mô hình tuyến tính:

- Lần huấn luyện đầu dùng Linear Regression thông thường cho kết quả hỏng: R^2 trên test là −2,68 triệu, MAE 449,6 tỷ và 45 dự đoán vượt 10.000 tỷ.


So sánh hiệu quả các mô hình (5-Fold CV)

Phiên bản kết quả: 32bb3360-a5f5c1ea | Ngày lưu (UTC): 2026-10-17T07:19:05.968075+00:00

	Mô hình	Test R^2	CV R^2 (mean)	CV R^2 (std)	CV MAE (tỷ)	Test MAE (tỷ)	CV RMSE (tỷ)	Test RMSE (tỷ)	CV MAPE (%)	Test MAPE (%)	Best
0	Linear Regression	-2684033.164	-3745030.428	±5715861.819	354.475	449.619	61335.778	61437.242	129.53	217.38	
1	Random Forest	0.711	0.660	±0.040	5.105	5.122	22.895	20.151	25.32	25.24	✓
2	Gradient Boosting	0.701	0.634	±0.065	5.206	5.224	23.751	20.494	25.55	25.51	

- Truy vết cho thấy dòng lớn nhất có diện tích 2.290 m² bị dự đoán trên 15 triệu tỷ đồng: diện tích thô bị ngoại suy quá xa, rồi phép `expm1` khuếch đại kết quả.
- Chỉ giới hạn diện tích tối đa không giải quyết được nguyên nhân gốc, và Ridge với alpha cố định rất lớn chỉ giảm triệu chứng. Giải pháp cuối cùng là áp `log1p` cho 5 biến số trước khi đưa vào Ridge, kèm `RidgeCV` tự chọn alpha.

2.4 Chỉ số đánh giá

R^2 trên giá gốc bị chi phối bởi vài căn rất đắt, nên nhóm báo cáo thêm:

- **MAE**: sai số tuyệt đối, tỷ VNĐ
- **Median APE**: trung vị sai số phần trăm, phản ánh căn nhà điển hình
- **Tỷ lệ dự đoán lệch không quá 20%**
- **R^2 trên thang log:** cùng thang với mục tiêu huấn luyện và so sánh với **baseline** luôn đoán giá trung vị của tập train.

3. Thiết kế chương trình/hệ thống

3.1 Luồng xử lý

1. **Đọc dữ liệu**: nạp 10 file parquet, lọc Hà Nội, chọn cột cần dùng.
2. **Làm sạch** theo quy tắc cố định từng dòng; bỏ loại "Đất".
3. **Chia train/test theo nhóm** tin trùng (80/20, `random_state=42`), rồi **khử trùng** riêng từng tập.
4. **Tiền xử lý** (fit chỉ trên train): điền trung vị, cột chỉ báo thiếu, mã hóa quận/phường out-of-fold, one-hot.
5. **Cross-validation 5-fold theo nhóm**: mỗi fold fit lại preprocessor, huấn luyện ba mô hình, đo các chỉ số.
6. **Chọn mô hình** theo CV, **đánh giá trên tập test**, tính baseline và sai số theo nhóm.
7. **Huấn luyện lại** mô hình tốt nhất trên toàn bộ dữ liệu đã khử trùng; **lưu artifact** (`model.pkl`, `preprocessor.pkl`) cùng phiên bản và thống kê dữ liệu.
8. **Ứng dụng Streamlit** đọc artifact, kiểm tra model và preprocessor cùng phiên bản, rồi phục vụ ba tab.

3.2 Cấu trúc mã nguồn

File	Vai trò
src/config.py	Hằng số: đường dẫn, ngưỡng làm sạch, danh sách cột
src/data_loader.py	Đọc parquet, lọc Hà Nội
src/cleaning.py	Làm sạch theo dòng, gắn cờ lỗi nhập liệu, khử trùng
src/splitting.py	Tạo khóa nhóm, chia train/test theo nhóm
src/preprocessing.py	Preprocessor có trạng thái: điền thiếu, mã hóa vị trí OOF, one-hot
src/training.py	Định nghĩa mô hình, cross-validation, chọn mô hình, refit
src/evaluation.py	Tính chỉ số và sai số theo nhóm
src/validation.py	Kiểm tra index, căn chỉnh nhãn, giao nhóm train/test
src/artifacts.py	Lưu và nạp artifact, gắn phiên bản
src/train_model.py	Chương trình huấn luyện chính
app.py	Ứng dụng Streamlit (3 tab)
tests/	44 unit test (rò rỉ dữ liệu, trường hợp biên, chỉ số, artifact, giao diện)

3.3 Kỹ thuật quan trọng

Chia theo nhóm để cùng một căn không xuất hiện ở cả train và test:

```
splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_pos, test_pos = next(splitter.split(cleaned_df, groups=groups))
```

Pipeline Ridge trên đặc trưng log, giải quyết lỗi ngoại suy của mô hình tuyến tính:

```
Pipeline([
    ("log", FunctionTransformer(log_numeric_columns)),
    ("scaler", StandardScaler()),
    ("model", RidgeCV(alphas=np.logspace(-2, 3, 12))),
])
```

4. Thư viện, framework và công nghệ sử dụng

Công nghệ	Mục đích
Python 3.13	Ngôn ngữ chính
pandas, NumPy	Xử lý và tính toán dữ liệu
PyArrow	Đọc file parquet, lọc theo tỉnh khi đọc
scikit-learn 1.9	Mô hình (Ridge, Random Forest, Gradient Boosting), GroupKFold, GroupShuffleSplit, đánh giá
Streamlit (từ 1.65)	Giao diện web
Plotly	Biểu đồ trong ứng dụng
unittest	44 bài kiểm thử tự động
Hugging Face	Nguồn dữ liệu
Git, GitHub	Quản lý mã nguồn

5. Kết quả thực hiện và minh họa sản phẩm

5.1 So sánh mô hình

Các chỉ số tính trên 131.539 căn của tập test, đơn vị tiền là tỷ VNĐ (giá trung vị toàn tập là 11,26 tỷ):

Mô hình	CV R^2 (5-fold)	Test R^2	R^2 log	MAE	Median APE	Lệch ≤ 20%
Random Forest (được chọn)	0,660 ± 0,040	0,711	0,851	5,12	17,4%	55,9%
Gradient Boosting	0,634 ± 0,065	0,701	0,850	5,22	18,0%	54,6%
RidgeCV (log feature)	0,573 ± 0,059	0,614	0,816	5,91	21,0%	48,0%
Baseline (đoán trung vị train)	—	—	—	12,07	49,3%	—

Nhận xét:

- Random Forest giảm MAE khoảng 58% so với baseline. Điểm CV và test không chênh lệch lớn; pipeline đồng thời kiểm soát rò rỉ bằng cách chia theo nhóm và fit preprocessing riêng trong từng fold.
- Ridge sau khi log hóa đạt R^2 log 0,816, cho thấy mô hình log-tuyến tính giải thích được phần lớn biến thiên giá trên thang log. Hai mô hình cây vẫn tốt hơn nhờ học quan hệ phi tuyến và tương

tác giữa các đặc trưng; Random Forest đạt Test R^2 0,711 so với 0,614 của Ridge và tỷ lệ lệch không quá 20% là 55,9% so với 48,0%.

- Các đặc trưng quan trọng nhất của Random Forest: diện tích (0,62), loại căn hộ chung cư (0,22), giá trung vị phường (0,07) và giá trung vị quận (0,04).

5.2 Sai số theo nhóm (Random Forest, tập test)

Khoảng giá thật	Số mẫu	Median APE	Lệch ≤ 20%
Dưới 3 tỷ	2.295	28,6%	39,6%
3–6 tỷ	19.121	14,4%	62,2%
6–10 tỷ	37.437	14,4%	63,8%
10–20 tỷ	39.563	17,9%	54,9%
Trên 20 tỷ	33.123	22,2%	45,4%

Mô hình chính xác nhất ở phân khúc 3–10 tỷ:

- Theo loại bất động sản, căn hộ chung cư tốt nhất (Median APE 12,9%, 68,8% căn lệch ≤ 20%), nhà phố 19,2%, biệt thự/nhà liền kề 16,4% và shophouse kém nhất (26,7%).
- Theo quận, Thanh Oai, Chương Mỹ và Thạch Thất cho tỷ lệ lệch ≤ 20% cao nhất (khoảng 75%, nhưng chỉ có 210–424 mẫu mỗi quận), còn Hoàn Kiếm thấp nhất (36,9%, MAE khoảng 31 tỷ với giá trung vị 32 tỷ)

Lưu ý: khi nhóm theo giá thật, dự đoán thường cao hơn ở nhóm rẻ và thấp hơn ở nhóm đắt (hồi quy về trung bình), nên bảng này dùng để biết mô hình yếu ở đâu, không để kết luận mô hình lệch trên từng căn.

5.3 Minh họa sản phẩm

Tab 1: "Tổng quan dữ liệu" trình bày phân phối giá và giá trung vị theo quận.

Tab 2: "Dự đoán giá" cung cấp biểu mẫu nhập thông tin và kết quả định giá kèm khoảng tham khảo.

Tab 3: "Đánh giá mô hình" hiển thị bảng so sánh mô hình và sai số theo nhóm.

Ví dụ dự đoán chạy trên mô hình Random Forest hiện hành (là đầu ra của mô hình, không phải giá giao dịch thực tế). Khoảng tham khảo bằng giá dự đoán nhân 1 ± 25,2% (MAPE của mô hình):

Bất động sản	Giá dự đoán	Khoảng tham khảo
Nhà, Cầu Giấy (Dịch Vọng), 50 m², 3 phòng ngủ, 4 tầng, mặt tiền 4 m	14,55 tỷ	10,88 – 18,22 tỷ
Nhà, Hoàn Kiếm, 60 m², 4 phòng ngủ, 5 tầng, mặt tiền 4,5 m	25,82 tỷ	19,30 – 32,34 tỷ
Căn hộ chung cư, Nam Từ Liêm, 75 m², 2 phòng ngủ	5,89 tỷ	4,40 – 7,38 tỷ
Biệt thự/nhà liền kề, Hà Đông, 200 m², 5 phòng ngủ, 4 tầng, mặt tiền 8 m	41,14 tỷ	30,76 – 51,52 tỷ

5.4 Kiểm thử

Bộ 44 unit test chạy thành công, bao gồm kiểm tra không có giao nhóm giữa train và test, preprocessing chỉ học từ train, công thức các chỉ số, độ ổn định của pipeline Ridge với căn diện tích rất lớn, và giao diện ứng dụng với cả artifact mới lẫn cũ.

6. Kết luận

Kết quả đạt được. Nhóm xây dựng được hệ thống dự đoán giá nhà Hà Nội hoàn chỉnh từ dữ liệu thô đến ứng dụng web. Mô hình Random Forest đạt R^2 0,711 trên giá gốc và 0,851 trên thang log, sai lệch trung vị 17,4%, và 55,9% dự đoán lệch không quá 20% so với giá chào bán. Điểm mạnh của đề tài là quy trình đánh giá trung thực (chia theo nhóm, khử trùng sau khi chia, mã hóa vị trí out-of-fold) và việc phát hiện, khắc phục lỗi mô hình tuyến tính.

Hạn chế.

- Hệ thống dự đoán giá chào bán, không phải giá giao dịch; giá giao dịch thực tế có thể khác giá đăng tin.
- Chỉ có 8 đầu vào, thiếu tọa độ, hướng nhà, pháp lý, độ rộng ngõ, nên sai số lớn ở khu vực giá cao như Hoàn Kiếm và ở nhà trên 20 tỷ (chỉ 45,4% căn lệch ≤ 20%). Nguyên nhân này là suy luận hợp lý, chưa được kiểm chứng bằng thí nghiệm.
- Tập test đã được lọc bằng quy tắc có dùng giá (đơn giá ngoài khoảng 3 triệu đến 4 tỷ/m^2), nên điểm số phản ánh dữ liệu đã làm sạch chứ không phải mọi tin đăng.
- Khử trùng chỉ là gần đúng: cùng một căn đăng lại với giá khác vẫn bị coi là hai nhóm.
- Khoảng tham khảo trên ứng dụng không phải khoảng tin cậy; siêu tham số của Random Forest và Gradient Boosting được đặt cố định, chưa tinh chỉnh.

Hướng phát triển. Bổ sung tọa độ và khoảng cách tới trung tâm; dùng quantile regression để có khoảng dự đoán có độ bao phủ kiểm chứng được; làm mượt mã hóa giá phường ít tin; tinh chỉnh siêu tham số; thu thập giá giao dịch thực tế; triển khai ứng dụng lên máy chủ.

7. Phân công công việc

Thành viên	MSSV	Công việc đã thực hiện
Võ Quang Huy	B24DCKH063	• Thu thập dataset • Thống nhất pipeline • Chọn mô hình
Nguyễn Hữu Nam	B24DCKH092	• Đưa ra giải pháp làm sạch dữ liệu (data_loader.py, cleaning.py) • Tiền xử lý, kiểm soát rò rỉ dữ liệu (preprocessing.py, validation.py) • Huấn luyện, đánh giá và lưu kết quả so sánh chỉ số
Lê Đình Phong	B24DCKH098	• Ứng dụng Streamlit (app.py, .streamlit) • Kiểm thử, README, báo cáo và slide

8. Khai báo sử dụng công cụ AI

Nhóm sử dụng hai công cụ AI trong quá trình thực hiện. Chi tiết từng công cụ được ghi trong các file agent/Claude và agent/ChatGPT của mã nguồn.

Công cụ	Mục đích sử dụng
Claude (Anthropic)	• Đọc và đánh giá mã nguồn, kết quả huấn luyện; • Chẩn đoán lỗi mô hình tuyến tính; viết và sửa evaluation.py (Median APE, tỷ lệ lệch, bias, R^2 log, RMSLE, sai số theo nhóm), tab "Đánh giá mô hình" trong app.py; • Viết các test test_metrics, test_app_metrics, test_ridge_pipeline; cập nhật requirements.txt • Hỗ trợ soạn bản nháp báo cáo
ChatGPT/Codex (OpenAI)	• Phân tích và sửa quy trình lấy mẫu cho EDA • Truy vết các dự đoán cực đoan của mô hình tuyến tính • Thử Ridge và tích hợp pipeline log1p + Ridge • Mở rộng hệ thống chỉ số • Cải thiện giao diện và theme Streamlit • Bổ sung test

Phụ lục. Chạy sản phẩm và tài nguyên nộp

Yêu cầu Python 3.12 trở lên; artifact hiện tại được huấn luyện bằng Python 3.13. Có thể chạy ứng dụng mà không cần huấn luyện lại.

Cấu trúc thư mục:

```
du_doan_gia_nha
│
├── data/
│   └── parquet_dataset/ shard_0000.parquet -> shard_0009.parquet
├── artifacts/
│   ├── model.pkl
│   └── preprocessor.pkl
├── results/
│   ├── metrics.json
│   └── evaluation.csv
├── src/
│   ├── __init__.py
│   ├── config.py              # constants/config
│   ├── data_loader.py         # đọc parquet
```

```
│   ├── cleaning.py          # clean + error flags + dedup
│   ├── splitting.py         # GroupShuffleSplit
│   ├── preprocessing.py     # impute + encoding + transform
│   ├── train_model.py
│   ├── training.py          # CV + train model
│   ├── evaluation.py        # metric tổng thể và sai số theo nhóm
│   ├── artifacts.py         # save/load model + preprocessor
│   └── validation.py        # kiểm tra input / required columns
├── tests/
├── app.py
├── requirements.txt
```

```
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m unittest discover -s tests
streamlit run app.py
```